DESARROLLO DE APLICACIONES II

Desplegar el Core

Módulo 9 · Core

20 de octubre de 2026

Contenido

Antes que nada: ahora es un solo proceso 3
Opción 1 — Railway (la más fácil) 3
Opción 2 — Gratis, en tres paneles 4
La clave de firma 4
Cargar los datos iniciales 5
Probarlo entero antes, en tu máquina 5
Checklist 5
Lo que falta decidir 6

> **La respuesta corta:** Railway, un proyecto con tres servicios (el Core, Postgres y RabbitMQ), ~5 USD por mes. Es lo único que tiene las tres piezas en el mismo lugar y conectadas solas. Si tiene que ser gratis, la combinación de abajo funciona pero son tres paneles distintos.

El Core necesita **tres cosas**: el proceso de la API, una base Postgres y un RabbitMQ. La que complica es RabbitMQ: casi ningún plan gratuito lo incluye.

Antes que nada: ahora es un solo proceso

Hasta ayer el Core eran **dos procesos**: el servidor de la API y el worker que consume `core.inbox`. Eso obligaba a pagar dos servicios en cualquier plataforma.

Con `RUN_WORKER=true` el servidor levanta los consumidores en su propio loop y alcanza con **un contenedor**.

```
RUN_WORKER=true uvicorn app.main:app
```

Verificado de punta a punta contra RabbitMQ real: se publica en `muni.inbox`, el proceso lo consume, lo rutea a los 4 módulos suscriptos y queda en `q.obras` con los headers `x-core-*`.

El costo de unirlos. Los consumidores comparten el event loop con las peticiones HTTP, no se escalan por separado, y si uno se cae se cae todo. Para el volumen de este TP no se nota. Si algún día hace falta separarlos, el worker sigue existiendo: `python -m app.workers.inbox_worker` con `RUN_WORKER=false`.

Opción 1 — Railway (la más fácil)

Unos 15 minutos, la mayoría esperando builds. Todo en un proyecto, los servicios se ven entre sí por red interna, y el deploy sale de un push.

Los pasos

1. **New Project** → **Deploy from GitHub repo**, elegí este repo.
2. **Settings** → **Root Directory:** `backend`. Este es el único paso que hay que hacer a mano y el que más se olvida: el repo tiene `backend/` y `docs/`, y sin esto Railway busca el Dockerfile en la raíz y no lo encuentra.
3. **+ New** → **Database** → **PostgreSQL.**
4. **+ New** → **Template** → **RabbitMQ.**
5. En el servicio del Core, pestaña **Variables**:

```
ENVIRONMENT=production
DEBUG=false
RUN_WORKER=true
SEED_ON_STARTUP=true
DATABASE_URL=${{Postgres.DATABASE_URL}}
RABBITMQ_URL=${{RabbitMQ.AMQP_URL}}
JWT_PRIVATE_KEY=<la clave, ver abajo>
CORS_ORIGINS=https://<tu-front>.vercel.app
```

Las dos `${{...}}` las resuelve Railway solo.

6. **Settings** → **Networking** → **Generate Domain.** Esa es la URL para los equipos.

Listo. El primer arranque siembra la base solo: 9 módulos, 73 tipos de evento, 16 barrios y los catálogos.

Dos cosas que ya no hacen falta

No hay que corregir la `DATABASE_URL`. Railway la entrega como `postgresql://` y SQLAlchemy necesita `postgresql+asyncpg://`. Antes eso rompía el arranque con `No module named 'psycopg2'`,

que no dice nada del problema real. Ahora el Core completa el driver solo, así que podés pegar la variable tal cual.

No hay que entrar por consola a correr el seed. Con `SEED_ON_STARTUP=true` lo hace el arranque. Es idempotente: los redeploys no duplican nada ni rotan secrets ya generados.

Costo: el plan Hobby son 5 USD/mes e incluye crédito de uso. Para tres servicios chicos alcanza.

Dónde se rompe

Síntoma	Causa
El build no encuentra el Dockerfile	Falta **Root Directory:** `backend`
`RuntimeError: JWT_PRIVATE_KEY es obligatoria`	No cargaste la clave
`/health/ready` dice `degraded`	La base está bien, el broker no: revisá `RABBITMQ_URL`
El front da error de CORS	`CORS_ORIGINS` no tiene el dominio del front

Opción 2 — Gratis, en tres paneles

Pieza	Dónde	Límite relevante
RabbitMQ	**CloudAMQP**, plan *Little Lemur*	1M mensajes/mes, **20 conexiones**, 100 colas
Postgres	**Neon**	siempre encendida, no se duerme
API	**Render**, free web service	**se duerme a los 15 min sin tráfico**
Frontend	**Vercel** o **Netlify**	—

Dos advertencias que importan de verdad:

Las 20 conexiones de CloudAMQP son el límite real, no las colas. Nuestra topología usa 15 colas y 7 exchanges con los 9 módulos: sobra lugar. Pero si los 9 equipos abren 2 conexiones cada uno más la nuestra, estamos justo en el borde. Si empiezan a ver `connection refused`, es eso.

El free de Render se duerme y eso rompe el consumidor. La API se despierta con el primer request, pero mientras duerme no consume de `core.inbox`. Los eventos no se pierden —la cola es durable y se drenan al despertar— pero las entregas se demoran lo que tarde en despertarse. Para una demo en vivo, es suficiente; para que los otros equipos prueben a cualquier hora, es molesto.

Para `render.yaml` ya está el blueprint en la raíz del repo.

La clave de firma

En producción el arranque falla sin `JWT_PRIVATE_KEY`. No es una advertencia, es un `RuntimeError`:

```
RuntimeError: JWT_PRIVATE_KEY es obligatoria en produccion: sin clave fija,
cada reinicio invalidaria todos los tokens emitidos.
```

Generala así:

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out core.key
```

Y pegá el contenido completo en la variable de entorno, con los `-----BEGIN-----` y `-----END-----`. Si la plataforma no acepta saltos de línea, podés poner `\n` literales: el Core los convierte.

`core.key` **no va al repo.** Si se filtra, cualquiera puede firmar un token diciendo que es admin.

Cargar los datos iniciales

Lo más simple es `SEED_ON_STARTUP=true` y olvidarse. Si preferís hacerlo a mano:

```
DATABASE_URL="postgresql+asyncpg://..." python -m app.seeds
```

Carga los 9 módulos con su cuenta, los 73 tipos de evento, y los barrios, zonas y catálogos globales. Es **idempotente**: correrlo dos veces no duplica nada ni rota los secrets ya generados.

Los secrets de máquina

Hay una diferencia importante entre los dos modos:

- **A mano**, el seed **imprime los secrets una sola vez**. Copialos ahí.
- **Al arranque**, **no los imprime a propósito**: stdout son los logs de la plataforma y quedarían ahí para siempre, a la vista de cualquiera con acceso al panel. En su lugar deja un aviso diciendo qué módulos tienen secret.

Para obtener el de un módulo, con la cuenta de admin:

```
POST /api/v1/modules/{nombre}/rotate-secret
```

Devuelve uno nuevo, una sola vez. **Entregáselo al equipo por un canal privado**, no por el chat del grupo: ahí queda en el historial de todos para siempre.

Probarlo entero antes, en tu máquina

```
docker compose up --build
```

Levanta Postgres, RabbitMQ y el Core con las mismas tres variables que necesita el deploy. Si funciona ahí, funciona desplegado.

- API: `http://localhost:8000/docs`
- RabbitMQ: `http://localhost:15672` (guest/guest)

Checklist

- ☐ `ENVIRONMENT=production` y `DEBUG=false`
- ☐ `JWT_PRIVATE_KEY` cargada (si falta, no arranca)
- ☐ `DATABASE_URL` con `postgresql+asyncpg://` , no `postgresql://`
- ☐ `RABBITMQ_URL` apuntando al broker gestionado (`amqps://` con TLS)
- ☐ `RUN_WORKER=true` , salvo que despliegues el worker aparte
- ☐ `CORS_ORIGINS` con el dominio del frontend, nunca `*`
- ☐ El seed corrido una vez
- ☐ `/health/ready` devuelve `status: up` (si dice `degraded` , la base está bien y el broker no)

Lo que falta decidir

Dos ambientes o uno. El TP pide Test y Producción. Con un solo proyecto por ambiente se duplica el costo; para un TP, un solo ambiente desplegado y el resto local es defendible, pero hay que decirlo y no prometer dos.

Si se hacen los dos, la separación va por **vhost de RabbitMQ** (`/test` y `/prod`), no por prefijo de cola: con prefijos, un error de configuración publica en producción sin que nada falle.

Las credenciales de los 9 equipos. Cuando esté desplegado hay que entregar a cada módulo su secret de máquina y su usuario de RabbitMQ, **uno por uno y por privado**. No por el chat del grupo.

El usuario AMQP por módulo no es opcional. Hoy, publicando por AMQP, el Core no valida que `sourceModule` coincida con nadie: cualquiera puede publicar diciendo que es otro. Eso se cierra con un usuario de RabbitMQ por módulo y verificando el `user-id` del mensaje. Es parte del deploy, no algo para después.